Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия

Дисциплина: «*Базы данных*»

Отчёт
По лабораторной работе №3
Вариант ***2691***

Выполнил:
Крамарь Михаил Романович
P3113

Проверил:
Остапенко Иван Вадимович

г. Санкт-Петербург
2026

Содержание

Текст задания 3
Исходная модель 4
Изначальные функциональные зависимости 5
Нормальные формы 6
BCNF 7
Денормализация 8
Триггер и связанная с ним функция 9
Выводы по работе 10

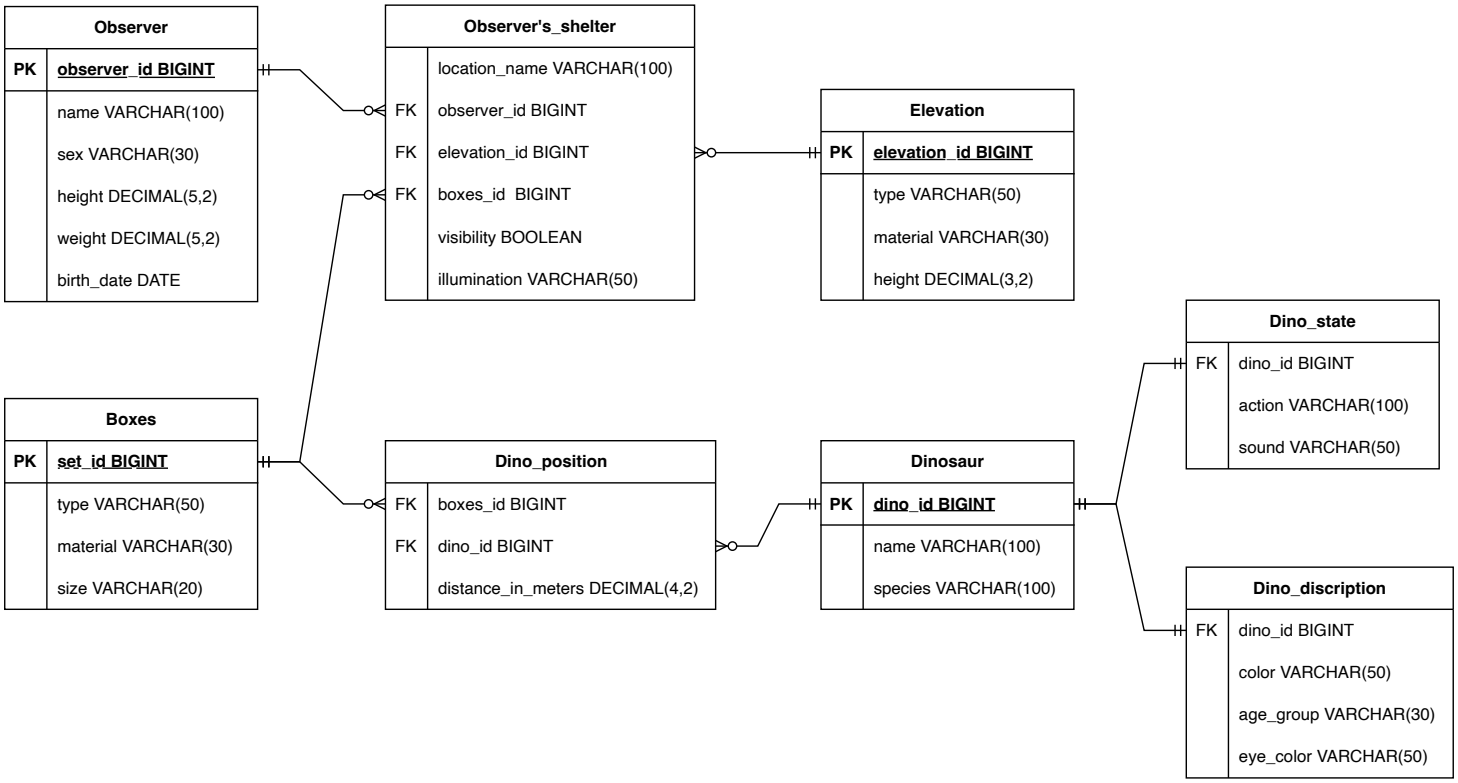
Текст задания
Лабораторная работа №3

Для отношений, полученных при построении предметной области из лабораторной работы №1, выполните следующие действия:

- Опишите функциональные зависимости для отношений полученной схемы (минимальное множество);
- Приведите отношения в 3NF (как минимум). Постройте схему на основеNF (как минимум).
- Опишите изменения в функциональных зависимостях, произошедшие после преобразования в 3NF (как минимум). Постройте схему на основеNF;
- Преобразуйте отношения в BCNF. Докажите, что полученные отношения представлены в BCNF. Если ваша схема находится уже в BCNF, докажите это;
- Какие денормализации будут полезны для вашей схемы? Приведите подробное описание.

Придумайте триггер и связанную с ним функцию, относящиеся к вашей предметной области, согласуйте их с преподавателем и реализуйте на языке PL/pgSQL.

Исходная модель



Изначальные функциональные зависимости

- Если в ***R*** (отношение или же таблица) атрибут ***A_2 функционально зависит*** от атрибута ***A_1***, то каждое значение ***A_1*** связано с одним значением ***A_2*** и определяет его.

- *Observer:* ***observer_id*** → *(name, sex, height, weight, birth_date)*
- *Boxes:* ***set_id*** → *(type, material, size)*
- *Dinosaur:* ***dino_id*** → *(name, species)*
- *Elevation:* ***elevation_id*** → *(type, material, height)*
- *Observer's_shelter:* ***observer_id, elevation_id, boxes_id*** → *(location_name, visibility, illumination)*
- *Dino_position:* ***dino_id, boxes_id*** → *distance_in_meters*
  Dino_state: ***dino_id*** → *(action, sound)*
- *Dino_description:* ***dino_id*** → *(color, age_group, eye_color)*

Нормальные формы

1NF: Отношение, на пересечении каждой строки и столбца — одно значение. Моя исходная модель соответствует 1NF, так как на пересечении и вправду всегда находится ровно одно значение.

2NF: 1) отношение в 1NF и 2) атрибуты, не входящие в первичный ключ, в ***полной функциональной зависимости*** от первичного ключа отношения. Моя модель соответствует 2NF, так как, как можно заметить, атрибуты, которые не входят в первичный ключ, находятся в полной функциональной зависимости от первичного ключа отношения.

3NF: отношение в 1) 1НФ и 2НФ и 2) все атрибуты, которые не входят в первичный ключ, не находятся в транзитивной функциональной зависимости от первичного ключа. Моя модель также удовлетворяет третьей нормальной форме, потому что все атрибуты, не входящие в первичный ключ, не находятся в транзитивной функциональной зависимости от первичного ключа отношения.

В качестве примера рассмотрим отношение ***Observer's_shelter***, в котором все неключевые атрибуты находятся в полной функциональной зависимости только от составного первичного ключа *(observer_id, elevation_id, boxes_id)*, несмотря на желание сказать, что освещение локации зависит от атрибута *location_name*, который, в свою очередь, по предположению зависит от нашего PK.

- ***Полная функциональная зависимость****: A_2 в полной функциональной зависимости от A_1, если $A_1 \rightarrow A_2$, но нет зависимостей вида $A_3 \rightarrow A_2$, где A_3 — подмножество A_1.*

*Итак, моя модель удовлетворяет как минимум **3NF!***

BCNF

- ***НФБК*** — отношение в НФБК, когда для всех функциональных зависимостей отношения выполняется условие: *детерминант — потенциальный ключ.*

Моя модель является отношением в нормальной форме Бойса-Когда, так как для каждой функциональной зависимости первичный ключ является детерминантом.

В случае отношений *Dinosaur, Observer, Elevation, Boxes* - это **id**, а в случае ассоциативных и характеристических сущностей FK является и PK, в ассоциациях PK - составной.

*Итак, моя модель удовлетворяет **BCNF!***

Денормализация

- Бывает, что для повышения производительности запросов производится *<u>денормализация</u>*: *несколько отношений объединяют в одно*;

Можно увеличить скорость выполнения некоторых запросов к базе данных за счёт сокращения количества сущностей, в особенности характеристических, так как мы можем просто добавить их атрибуты в ту стержневую сущность, которую они описывают *(**Dino_state** и **Dino_description** для **Dinosaur**)*. Мы избавимся от лишних JOIN'ов в будущем, но возможно получим слишком «широкие» таблицы. Также можно заменить составной ключ из трёх полей на один - ***shelter_id*** в отношении ***Observer's_shelter***, потому что составные PK усложняют описание запросов. Это обычное упрощение структуры, которое должно немного повысить производительность, СУБД будет сравнивать только одно число, а это физически быстрее для процессора.

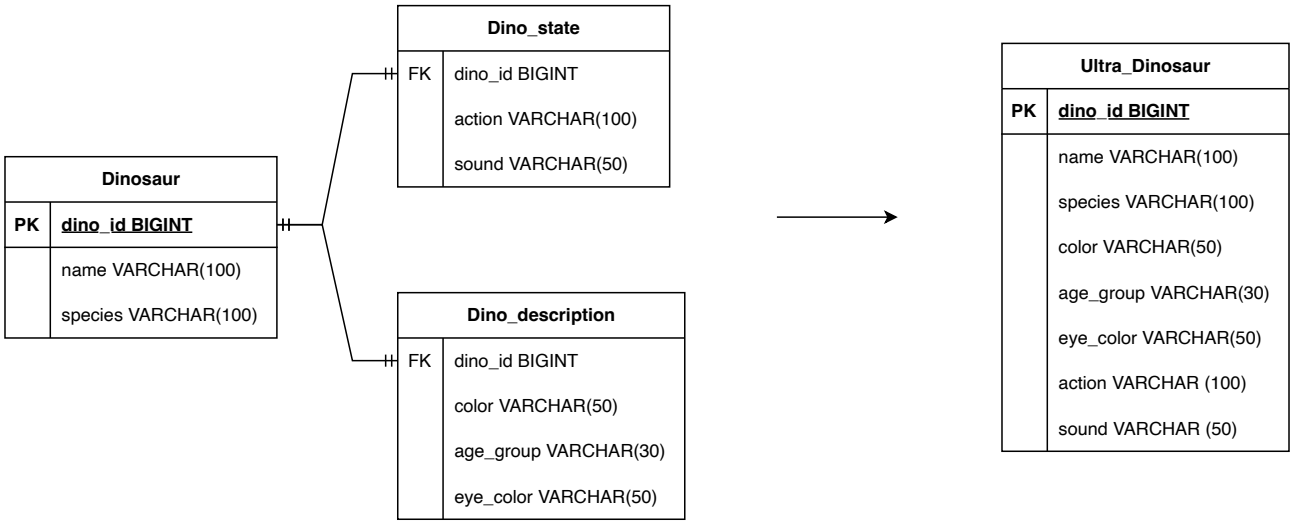


Рис. 1 преобразование к Ultra_Dinosaur

Триггер и связанная с ним функция

- Автоматическое обновление статуса динозавра.

Когда мы добавляем запись в отношение *Dino_position* и видим, что расстояние между набором ящиков и динозавром меньше *2* метров, триггер должен автоматически обновить статус динозавра (*action*) в таблице *Dino_state* на ‘**ATTACKING**’.

- ***Функция и триггер:***

```
-- наша функция
CREATE OR REPLACE FUNCTION s502511.checkDinoAction()
    RETURNS TRIGGER AS $$
    BEGIN
        -- NEW - это запись строки, которая наша база "держит" перед тем, как вставить в таблицу
        IF NEW.distance_in_meters < 2.00 THEN
            UPDATE s502511.dino_state SET action = 'ATTACKING'
            WHERE dino_id = NEW.dino_id;
            -- выводим сообщение в консоль (Messages)
            RAISE NOTICE 'Опасность! Динозавр % слишком близко к ящикам!', NEW.dino_id;
        END IF;

        RETURN NEW; -- формальность, так как требуется RETURN
    END;
    $$ LANGUAGE plpgsql;

-- тут создаём сам триггер, который сработает уже после вставки или обновления таблицы Dino_position
CREATE OR REPLACE TRIGGER dinoStatusAlert
    AFTER INSERT OR UPDATE ON s502511.dino_position
    -- теперь для каждой строки выполняем функцию
    FOR EACH ROW
    EXECUTE FUNCTION s502511.checkDinoAction();

-- пример выполнения
UPDATE s502511.dino_position SET distance_in_meters = 1.34
WHERE dino_id = 1 OR dino_id = 5;

SELECT * FROM s502511.dino_state;
```

Выводы по работе

В ходе выполнения данной лабораторной работы я познакомился с понятием нормализации и деморализации, функциональной зависимостью и различными видами нормальных форм. Я ознакомился с процедурным языком PL/pgSQL и придумал триггер, а также связанную с ним функцию, относящиеся к моей предметной области.